

Prefix Sum

1. Overview

Prefix sum stores cumulative sums so that any range sum can be answered in $O(1)$ after $O(n)$ preprocessing.

2. Intuition

Instead of adding $a[1] + \dots + a[r]$ every time, keep a running total up to every index. The sum from 1 to r is total up to r minus total before 1 . Like checking monthly expenses from a bank statement: subtract the balance before the month from the balance after the month.

3. When to Use It

- Range sum queries on static arrays.
- Trigger phrases: "sum from L to R", "multiple queries", "subarray sum", "count subarrays with sum K".
- 2D grid sum queries using 2D prefix sum.

4. When Not to Use It

- Frequent point/range updates: use Fenwick tree, segment tree, or difference array.
- Need min/max queries: use sparse table/segment tree.
- Only one query: direct loop may be simpler.

5. Core Concepts

- `prefix[i]`: sum of first i elements.
- 1-based prefix array avoids off-by-one: $\text{sum}(1, r) = \text{pref}[r + 1] - \text{pref}[1]$.
- Use `long long` for large values.

6. Step-by-Step Algorithm

1. Create `pref` of size $n + 1$, initialized with 0.
2. For each index i , set $\text{pref}[i + 1] = \text{pref}[i] + a[i]$.
3. For a query $[1, r]$, return $\text{pref}[r + 1] - \text{pref}[1]$.

7. Dry Run

Array: `[2, 4, -1, 3]`

i	a[i]	pref[i + 1]
0	2	2
1	4	6
2	-1	5
3	3	8

Query $[1, 3]: \text{pref}[4] - \text{pref}[1] = 8 - 2 = 6$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> buildPrefix(const vector<int>& a) {
    int n = (int)a.size();
    vector<long long> pref(n + 1, 0);
    for (int i = 0; i < n; i++) pref[i + 1] = pref[i] + a[i];
    return pref;
}

long long rangeSum(const vector<long long>& pref, int l, int r) {
    return pref[r + 1] - pref[l];
}

int main() {
    vector<int> a = {2, 4, -1, 3};
    auto pref = buildPrefix(a);
    cout << rangeSum(pref, 1, 3) << "\n"; // 6
}
```

pYTHON IMPLEMENTATION

```
def build_prefix(a):
    pref = [0]
    for x in a:
        pref.append(pref[-1] + x)
    return pref

def range_sum(pref, l, r):
    return pref[r + 1] - pref[l]

a = [2, 4, -1, 3]
pref = build_prefix(a)
print(range_sum(pref, 1, 3)) # 6
```

10. Code Explanation

$\text{pref}[0]$ represents sum before the array starts. Each next prefix extends the previous sum by one element. Range query removes the part before l , leaving only $[l, r]$.

11. Complexity Analysis

Operation	Time	Space
Preprocessing	$O(n)$	$O(n)$
Query	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Static range sum	Many <code>[l, r]</code> queries	Prefix sum	GFG Range Sum Queries
Subarray sum K	Count subarrays with target	Prefix + hash map	LeetCode 560
Equilibrium index	Left sum equals right sum	Prefix/total sum	GFG Equilibrium Point

13. Common Mistakes

- Mixing 0-based and 1-based formulas.
- Using `int` when sum can overflow.
- Forgetting that prefix sum works with negative numbers too.

14. Edge Cases

- Empty array.
- Single element.
- Negative values.
- Large values.
- Query covering entire array.

15. Variations

- 2D prefix sum for matrix rectangles.
- Prefix XOR for XOR range queries.
- Prefix counts for character/frequency queries.

16. Related Algorithms/Data Structures

- Fenwick tree: prefix sums with updates.
- Difference array: efficient range updates.
- Hashing with arrays: prefix states stored for counting.

17. Practice Problems

Easy

- Range Sum Query Immutable, LeetCode, static prefix query, Easy.
- Equilibrium Point, GFG, prefix/total sum, Easy.

Medium

- Subarray Sum Equals K, LeetCode, prefix + hash map, Medium.
- Product of Array Except Self, LeetCode, prefix/suffix products, Medium.
- CSES Static Range Sum Queries, CSES, prefix sums, Medium.

Hard

- Count of Range Sum, LeetCode, prefix + merge sort/BIT, Hard.
- Maximum Sum Rectangle, GFG, 2D prefix + Kadane, Hard.

18. Interview Explanation

Prefix sum precomputes cumulative totals so a range sum becomes subtraction of two prefix values. It is ideal when the array is static and there are many range sum queries.

19. Revision Notes

- Key idea: cumulative sum.
- Formula: $\text{sum}(l, r) = \text{pref}[r + 1] - \text{pref}[l]$.
- Complexity: build $O(n)$, query $O(1)$.
- Trap: overflow and indexing.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Static range sums	Build/query	$O(n)/O(1)$	$\text{pref}[i+1] = \text{pref}[i] + a[i]$	empty, negatives, large sums

Suffix Sum

1. Overview

Suffix sum stores cumulative sums from the right side of the array, allowing quick answers about suffixes and right-side contributions.

2. Intuition

Prefix sum answers "what is before me?" Suffix sum answers "what is after me?" Build totals backwards so every index knows the sum from itself to the end.

3. When to Use It

- Need right-side sums.
- Need compare left and right parts.
- Trigger phrases: "elements after index", "right side sum", "split array".

4. When Not to Use It

- For only left-to-right range queries, prefix sum is more natural.
- For changing arrays, use Fenwick/segment tree.
- If only total sum is enough, suffix array is unnecessary.

5. Core Concepts

- $\text{suf}[i] = a[i] + a[i + 1] + \dots + a[n - 1]$.
- $\text{suf}[n] = 0$ helps represent empty suffix.
- Range sum can also be $\text{suf}[l] - \text{suf}[r + 1]$.

6. Step-by-Step Algorithm

1. Create suf of size $n + 1$ with zeros.
2. Iterate from $n - 1$ to 0 .
3. Set $\text{suf}[i] = a[i] + \text{suf}[i + 1]$.
4. Use $\text{suf}[i + 1]$ for sum strictly after i .

7. Dry Run

Array: [5, 1, 2, 4]

i	a[i]	suf[i]
3	4	4
2	2	6
1	1	7
0	5	12

Sum after index 1 is $\text{suf}[2] = 6$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> buildSuffix(const vector<int>& a) {
    int n = (int)a.size();
    vector<long long> suf(n + 1, 0);
    for (int i = n - 1; i >= 0; i--) suf[i] = a[i] + suf[i + 1];
    return suf;
}

int main() {
    vector<int> a = {5, 1, 2, 4};
    auto suf = buildSuffix(a);
    cout << suf[2] << "\n"; // 6
}
```

pYTHON IMPLEMENTATION

```
def build_suffix(a):
    suf = [0] * (len(a) + 1)
    for i in range(len(a) - 1, -1, -1):
        suf[i] = a[i] + suf[i + 1]
    return suf
```

10. Code Explanation

The loop runs backward because `suf[i]` depends on `suf[i + 1]`. The extra last zero represents "no elements after the end".

11. Complexity Analysis

Operation	Time	Space
Build	$O(n)$	$O(n)$
Suffix query	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Split score	left score + right score	Prefix + suffix	LeetCode 1422
Products except self	product left and right	Prefix/suffix products	LeetCode 238
Right contribution	count/sum after index	Suffix totals	Codeforces array scoring

13. Common Mistakes

- Forgetting `suf[n] = 0`.
- Using `suf[i]` when you need elements strictly after `i`, which is `suf[i + 1]`.
- Overflow in suffix totals.

14. Edge Cases

- Last index.
- Single element.
- Negative values.
- Query for empty suffix.

15. Variations

- Suffix minimum/maximum.
- Suffix product.
- Suffix count of characters.

16. Related Algorithms/Data Structures

- Prefix sum: left-side cumulative data.
- Prefix min/max: best value before an index.
- Product except self: combines prefix and suffix.

17. Practice Problems

Easy

- Find Pivot Index, LeetCode, prefix/suffix balance, Easy.
- Left and Right Sum Differences, LeetCode, suffix sums, Easy.

Medium

- Product of Array Except Self, LeetCode, prefix/suffix product, Medium.
- Minimum Value to Get Positive Step by Step Sum, LeetCode, suffix reasoning alternative, Medium.

Hard

- Trapping Rain Water, LeetCode, prefix/suffix max, Hard.
- Maximum Sum of Three Non-Overlapping Subarrays, LeetCode, prefix/suffix best, Hard.

18. Interview Explanation

Suffix sum is the mirror of prefix sum. It precomputes cumulative values from the right so right-side sums or split comparisons are answered immediately.

19. Revision Notes

- Build backward.
- `suf[i]` includes `a[i]`.
- `suf[i + 1]` excludes `a[i]`.
- Use `long long`.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Right-side sums	Build/query	O(n)/O(1)	<code>suf[i]=a[i]+suf[i+1]</code>	last index, empty suffix

Difference Array

1. Overview

A difference array supports many range increment updates in O(1) each, then reconstructs the final array using prefix sum.

2. Intuition

To add x to $[l, r]$, mark "start adding x at l " and "stop adding x after r ". A prefix scan later applies all active changes.

3. When to Use It

- Multiple range update operations.
- Need final array, not online queries after every update.
- Trigger phrases: "add X to all elements from L to R ", "range increments", "after all operations".

4. When Not to Use It

- Need query results between updates: use segment tree/Fenwick with lazy logic.
- Need range min/max updates with complex constraints.
- Very small number of updates where direct loop is simpler.

5. Core Concepts

- $\text{diff}[0] = a[0], \text{diff}[i] = a[i] - a[i - 1]$.
- Range add: $\text{diff}[l] += x, \text{diff}[r + 1] -= x$ if valid.
- Prefix of diff gives final array.

6. Step-by-Step Algorithm

1. Create diff of size $n + 1$ initialized with zero.
2. For each update (l, r, x) , do $\text{diff}[l] += x, \text{diff}[r + 1] -= x$.
3. Scan from left to right accumulating current value.
4. Store current value as final array element.

7. Dry Run

$n = 5$, updates: add 3 to $[1, 3]$, add 2 to $[2, 4]$.

Step	diff
start	$[0, 0, 0, 0, 0, 0]$
+3 [1,3]	$[0, 3, 0, 0, -3, 0]$
+2 [2,4]	$[0, 3, 2, 0, -3, -2]$

Prefix scan: $[0, 3, 5, 5, 2]$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> applyRangeAdds(int n, const vector<tuple<int,int,int>>& updates)
{
    vector<long long> diff(n + 1, 0), result(n);
```

```

    for (auto [l, r, value] : updates) {
        diff[l] += value;
        if (r + 1 < n) diff[r + 1] -= value;
    }
    long long current = 0;
    for (int i = 0; i < n; i++) {
        current += diff[i];
        result[i] = current;
    }
    return result;
}

int main() {
    vector<tuple<int,int,int>> updates = {{1, 3, 3}, {2, 4, 2}};
    for (long long x : applyRangeAdds(5, updates)) cout << x << " ";
}

```

PYTHON IMPLEMENTATION

```

def apply_range_adds(n, updates):
    diff = [0] * (n + 1)
    for l, r, value in updates:
        diff[l] += value
        if r + 1 < n:
            diff[r + 1] -= value
    ans, current = [], 0
    for i in range(n):
        current += diff[i]
        ans.append(current)
    return ans

```

10. Code Explanation

Each update changes only two boundary markers. During reconstruction, `current` contains all range updates currently active at index `i`.

11. Complexity Analysis

Operation	Time	Space
Each update	$O(1)$	$O(1)$
Rebuild	$O(n)$	$O(n)$
Total for q updates	$O(n + q)$	$O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Range addition	many range increments	Difference array	LeetCode 370
Flight bookings	seats added over intervals	Difference array	LeetCode 1109
Imos method	intervals on line	Difference + prefix	AtCoder Imos tasks

13. Common Mistakes

- Updating `diff[r]` instead of `diff[r + 1]`.
- Allocating only `n` and writing out of bounds.
- Expecting it to answer online queries.

14. Edge Cases

- Update starts at `0`.
- Update ends at `n - 1`.
- Negative increments.
- Overlapping updates.

15. Variations

- 2D difference array for rectangle updates.
- Difference on compressed coordinates.
- Difference for interval coverage counts.

16. Related Algorithms/Data Structures

- Prefix sum reconstructs the final result.
- Sweep line is a coordinate-based difference array.
- Segment tree handles online range updates/queries.

17. Practice Problems

Easy

- Range Addition, LeetCode, difference array, Easy.
- Car Pooling, LeetCode, interval capacity, Easy/Medium.

Medium

- Corporate Flight Bookings, LeetCode, range additions, Medium.
- Shifting Letters II, LeetCode, signed difference, Medium.

Hard

- My Calendar III, LeetCode, sweep/difference map, Hard.
- CSES Polynomial Queries, CSES, advanced difference idea, Hard.

18. Interview Explanation

For many range updates, I record only where an increment starts and where it stops. A final prefix pass converts those boundary changes into actual values.

19. Revision Notes

- Range add: `diff[l] += x, diff[r + 1] -= x`.
- Reconstruct with prefix sum.
- Offline final-array technique.
- Trap: boundary at `r + 1`.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Many offline range adds	update/rebuild	$O(1)/O(n)$	boundary marks	<code>r=n-1</code> , overlaps

Kadane's Algorithm

1. Overview

Kadane's algorithm finds the maximum subarray sum in $O(n)$.

2. Intuition

At each index, decide: extend the previous subarray or start fresh here. If the previous sum hurts you, drop it. A negative running sum is like carrying debt into a new business deal.

3. When to Use It

- Maximum sum contiguous subarray.
- Trigger phrases: "maximum subarray", "contiguous segment", "best sum".
- 2D maximum rectangle after compressing rows/columns.

4. When Not to Use It

- Non-contiguous subsequence: use different DP/greedy.
- Need exact length/window: use sliding window/prefix.
- Need all subarrays: Kadane gives best, not enumeration.

5. Core Concepts

- `current`: best subarray ending at current index.
- `best`: best subarray seen so far.
- Works with all-negative arrays if initialized to first element.

6. Step-by-Step Algorithm

1. Set `current = best = a[0]`.

2. For each next element x , set $\text{current} = \max(x, \text{current} + x)$.
3. Set $\text{best} = \max(\text{best}, \text{current})$.
4. Return best .

7. Dry Run

Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

x	current	best
-2	-2	-2
1	1	1
-3	-2	1
4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

Answer: 6.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long maxSubarraySum(const vector<int>& a) {
    long long current = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        current = max<long long>(a[i], current + a[i]);
        best = max(best, current);
    }
    return best;
}

int main() {
    vector<int> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    cout << maxSubarraySum(a) << "\n"; // 6
}
```

pYTHON IMPLEMENTATION

```
def max_subarray_sum(a):
    current = best = a[0]
    for x in a[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best
```

10. Code Explanation

`current + a[i]` means extend. `a[i]` means restart at this element. `best` records the best ending position found.

11. Complexity Analysis

Operation	Time	Space
Scan	O(n)	O(1)

12. Common Patterns

Pattern	Identify	Approach	Examples
Max subarray	contiguous max sum	Kadane	LeetCode 53
Circular max	wrap allowed	max Kadane + min Kadane	LeetCode 918
Max rectangle	matrix max sum	row compression + Kadane	GFG Max Sum Rectangle

13. Common Mistakes

- Initializing `best = 0`, which fails for all-negative arrays.
- Applying to non-contiguous subsequences.
- Forgetting to track start/end when asked for indices.

14. Edge Cases

- Single element.
- All negative.
- All positive.
- Contains zeros.
- Large sums.

15. Variations

- Minimum subarray sum.
- Circular maximum subarray.
- Kadane with indices.
- 2D Kadane.

16. Related Algorithms/Data Structures

- Prefix minimum can also solve max subarray.
- Sliding window works when constraints are non-negative/fixed-size.
- DP generalizes Kadane.

17. Practice Problems

Easy

- Maximum Subarray, LeetCode, Kadane, Easy/Medium.
- Maximum Product Subarray basic variants, GFG, Kadane-like, Easy.

Medium

- Maximum Sum Circular Subarray, LeetCode, max/min Kadane, Medium.
- Best Time to Buy and Sell Stock, LeetCode, Kadane on differences, Medium.

Hard

- Maximum Sum Rectangle, GFG, 2D Kadane, Hard.
- Maximum Subarray Sum with One Deletion, LeetCode, DP Kadane, Hard.

18. Interview Explanation

Kadane keeps the best subarray ending at each index. If extending the previous sum is worse than starting at the current element, I restart. The maximum over these states is the answer.

19. Revision Notes

- `current = max(x, current + x)`.
- Initialize with `a[0]`.
- $O(n)$, $O(1)$.
- Trap: all-negative arrays.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Max contiguous sum	scan	$O(n)$	extend or restart	all negative, one element

Two Pointers

1. Overview

Two pointers uses two indices moving through an array to reduce nested loops, often from $O(n^2)$ to $O(n)$ or $O(n \log n)$.

2. Intuition

Instead of trying every pair blindly, use sorted order or monotonic movement. If one pointer move makes the answer too small or too large, move the other pointer accordingly.

3. When to Use It

- Sorted arrays.
- Pair/triplet sums.
- Removing duplicates.
- Trigger phrases: "two sorted arrays", "find pair", "in-place", "contiguous window".

4. When Not to Use It

- Array has no ordering/monotonic property and cannot be sorted.
- Need arbitrary range updates.
- Negative numbers can break variable-size sliding-window assumptions.

5. Core Concepts

- Opposite pointers: `left = 0, right = n - 1`.
- Same-direction pointers: both move forward.
- Each pointer should move at most $O(n)$ times.

6. Step-by-Step Algorithm

1. Identify what pointer movement means.
2. Initialize pointers.
3. Check current state.
4. Move the pointer that can improve the state.
5. Stop when pointers cross or reach the end.

7. Dry Run

Find pair sum 9 in sorted `[1,2,4,7,11]`.

left	right	sum	action
0	4	12	too large, right--
0	3	8	too small, left++
1	3	9	found

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

pair<int,int> twoSumSorted(const vector<int>& a, int target) {
    int left = 0, right = (int)a.size() - 1;
    while (left < right) {
```

```

        long long sum = (long long)a[left] + a[right];
        if (sum == target) return {left, right};
        if (sum < target) left++;
        else right--;
    }
    return {-1, -1};
}

int main() {
    vector<int> a = {1,2,4,7,11};
    auto [i, j] = twoSumSorted(a, 9);
    cout << i << " " << j << "\n";
}

```

pYTHON IMPLEMENTATION

```

def two_sum_sorted(a, target):
    left, right = 0, len(a) - 1
    while left < right:
        s = a[left] + a[right]
        if s == target:
            return left, right
        if s < target:
            left += 1
        else:
            right -= 1
    return -1, -1

```

10. Code Explanation

Because the array is sorted, increasing **left** increases the sum and decreasing **right** decreases it. That lets us discard many pairs safely.

11. Complexity Analysis

Operation	Time	Space
Sorted two-pointer scan	$O(n)$	$O(1)$
If sorting required	$O(n \log n)$	$O(1)$ or $O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Pair in sorted array	target sum	opposite pointers	LeetCode 167
Remove duplicates	compact array	slow/fast pointers	LeetCode 26
Container area	maximize width/height	opposite pointers	LeetCode 11

13. Common Mistakes

- Sorting when original indices must be preserved without storing them.
- Moving both pointers incorrectly.
- Infinite loop by not moving a pointer.

14. Edge Cases

- Empty array.
- One element.
- Duplicates.
- Negative values.
- No valid pair.

15. Variations

- Fast/slow pointer.
- Three pointers for Dutch national flag.
- Two pointers after sorting for 3Sum/4Sum.

16. Related Algorithms/Data Structures

- Sliding window is a two-pointer pattern for contiguous ranges.
- Sorting + greedy often enables two pointers.
- Hashing solves unsorted pair lookup.

17. Practice Problems

Easy

- Two Sum II, LeetCode, opposite pointers, Easy.
- Remove Duplicates from Sorted Array, LeetCode, slow/fast, Easy.

Medium

- 3Sum, LeetCode, sort + two pointers, Medium.
- Container With Most Water, LeetCode, greedy pointer move, Medium.

Hard

- Trapping Rain Water, LeetCode, two pointers/prefix max, Hard.
- Minimum Window Substring, LeetCode, sliding two pointers, Hard.

18. Interview Explanation

Two pointers works when moving one pointer has predictable effect. I use that monotonic behavior to discard many candidates while scanning the array only once.

19. Revision Notes

- Need a reason pointer movement is safe.
- Opposite pointers for sorted pair problems.
- Fast/slow for in-place compaction.
- Trap: losing original indices after sorting.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Pair/window/compaction	move indices	$O(n)$	monotonic pointer moves	duplicates, no answer

Sliding Window

1. Overview

Sliding window maintains a contiguous subarray while expanding and shrinking its boundaries.

2. Intuition

Imagine a window over the array. Move the right edge to include new elements; move the left edge to remove old elements when the window becomes invalid.

3. When to Use It

- Contiguous subarray/substring.
- Fixed-size window.
- Variable-size window with monotonic validity.
- Trigger phrases: "longest/shortest subarray", "at most K", "consecutive".

4. When Not to Use It

- Subsequence, not subarray.
- Negative numbers can break sum-based monotonic windows.
- Need non-contiguous choices: use DP/greedy.

5. Core Concepts

- Fixed window: add right, remove left once size exceeds k.
- Variable window: expand right, shrink while invalid.
- Maintain enough state: sum, counts, distinct count, max via deque.

6. Step-by-Step Algorithm

1. Set `left = 0`.
2. Iterate `right` from `0` to `n - 1`.
3. Add `a[right]` to state.
4. While window invalid, remove `a[left]` and increment `left`.

5. Update answer when valid.

7. Dry Run

Longest subarray with sum ≤ 5 , array $[1, 2, 1, 3, 1]$.

right	add	sum	left	action	best
0	1	1	0	valid	1
1	2	3	0	valid	2
2	1	4	0	valid	3
3	3	7	0	remove 1,2	2
4	1	5	2	valid	3

Answer: 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int longestSumAtMostK(const vector<int>& a, int k) {
    int left = 0, best = 0;
    long long sum = 0;
    for (int right = 0; right < (int)a.size(); right++) {
        sum += a[right];
        while (left <= right && sum > k) {
            sum -= a[left];
            left++;
        }
        best = max(best, right - left + 1);
    }
    return best;
}

int main() {
    vector<int> a = {1, 2, 1, 3, 1};
    cout << longestSumAtMostK(a, 5) << "\n";
}
```

pYTHON IMPLEMENTATION

```
def longest_sum_at_most_k(a, k):
    left = best = 0
    total = 0
    for right, x in enumerate(a):
        total += x
```

```

    while left <= right and total > k:
        total -= a[left]
        left += 1
    best = max(best, right - left + 1)
    return best

```

10. Code Explanation

The state is the current window sum. Since all values are non-negative, removing from the left is the correct way to reduce an invalid sum.

11. Complexity Analysis

Operation	Time	Space
Window scan	$O(n)$	$O(1)$
With frequency map	$O(n)$ average	$O(k/\text{distinct})$

12. Common Patterns

Pattern	Identify	Approach	Examples
Fixed size	exactly k elements	running sum	LeetCode 643
At most K distinct	longest valid window	freq map	LeetCode 904
Minimum covering	shortest window containing target	expand/shrink	LeetCode 76

13. Common Mistakes

- Using sum window with negative numbers.
- Updating answer before restoring validity.
- Forgetting to decrease frequency and distinct counts.

14. Edge Cases

- $k = 0$.
- Empty array.
- All values too large.
- All values valid.
- Duplicates in frequency windows.

15. Variations

- Monotonic deque for sliding max/min.
- At most K converted to exactly K: $\text{atMost}(k) - \text{atMost}(k - 1)$.
- Binary search + window for feasibility.

16. Related Algorithms/Data Structures

- Two pointers: sliding window is contiguous two pointers.
- Prefix sum: handles negative subarray sums better.
- Deque: max/min inside window.

17. Practice Problems

Easy

- Maximum Average Subarray I, LeetCode, fixed window, Easy.
- Contains Duplicate II, LeetCode, window set, Easy.

Medium

- Longest Substring Without Repeating Characters, LeetCode, frequency window, Medium.
- Fruit Into Baskets, LeetCode, at most two types, Medium.
- Subarrays with K Different Integers, LeetCode, atMost trick, Medium/Hard.

Hard

- Minimum Window Substring, LeetCode, covering window, Hard.
- Sliding Window Maximum, LeetCode, monotonic deque, Hard.

18. Interview Explanation

Sliding window keeps a valid contiguous range and updates it incrementally. Since each index enters and leaves at most once, the scan is linear.

19. Revision Notes

- Expand right.
- Shrink left while invalid.
- Update answer at the right time.
- Trap: negative numbers break sum monotonicity.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Contiguous ranges	expand/shrink	$O(n)$	maintain window state	invalid all, duplicates

Sorting + Greedy on Arrays

1. Overview

Sorting + greedy solves array problems by arranging data so that the locally best choice becomes globally safe.

2. Intuition

Sorting reveals order. Once ordered, you can pick earliest finishing intervals, smallest required resource, largest profit, or match extremes using a rule you can justify.

3. When to Use It

- Intervals, scheduling, pairing, minimizing/maximizing cost.
- Trigger phrases: "minimum number", "maximum tasks", "can arrange", "non-overlapping".
- When local choice has an exchange argument.

4. When Not to Use It

- If local optimal choice can block a better future and no proof exists.
- If original order is mandatory.
- If problem requires exploring combinations: use DP/backtracking.

5. Core Concepts

- Sort key matters: by end time, start time, value, ratio, etc.
- Greedy choice must be provably safe.
- Exchange argument: replace an optimal choice with greedy choice without worsening answer.

6. Step-by-Step Algorithm

1. Identify what should be optimized.
2. Choose a sorting key that exposes the safest local choice.
3. Sort.
4. Iterate and apply greedy rule.
5. Prove why each choice does not hurt future choices.

7. Dry Run

Intervals: `[[1,3],[2,4],[3,5]]`, select max non-overlapping.

Sort by end: same order.

interval	lastEnd	action	count
[1,3]	-inf	take	1
[2,4]	3	skip	1
[3,5]	3	take if touching allowed	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int maxNonOverlapping(vector<pair<int,int>> intervals) {
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
```

```

        return a.second < b.second;
    });
    int count = 0;
    int lastEnd = INT_MIN;
    for (auto [start, end] : intervals) {
        if (start >= lastEnd) {
            count++;
            lastEnd = end;
        }
    }
    return count;
}

int main() {
    vector<pair<int,int>> intervals = {{1,3},{2,4},{3,5}};
    cout << maxNonOverlapping(intervals) << "\n";
}

```

pYTHON IMPLEMENTATION

```

def max_non_overlapping(intervals):
    intervals.sort(key=lambda x: x[1])
    count, last_end = 0, float("-inf")
    for start, end in intervals:
        if start >= last_end:
            count += 1
            last_end = end
    return count

```

10. Code Explanation

Sorting by end time keeps future space maximum. Taking the interval that ends earliest leaves the most room for later intervals.

11. Complexity Analysis

Operation	Time	Space
Sort	$O(n \log n)$	$O(1)$ to $O(n)$
Greedy scan	$O(n)$	$O(1)$
Overall	$O(n \log n)$	$O(1)$ to $O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Activity selection	max non-overlap	sort by end	GFG Activity Selection

Pattern	Identify	Approach	Examples
Assign cookies	match small needs first	sort both arrays	LeetCode 455
Minimum arrows	burst intervals	sort by end	LeetCode 452

13. Common Mistakes

- Sorting by wrong field.
- Not handling equality correctly for touching intervals.
- Assuming greedy without proof.

14. Edge Cases

- Equal start/end.
- Empty intervals.
- Negative coordinates.
- Duplicates.
- Already sorted/reverse sorted.

15. Variations

- Sort by deadline for scheduling.
- Sort by ratio for fractional knapsack.
- Sort then two pointers for pairing.

16. Related Algorithms/Data Structures

- Merge intervals also sorts by start.
- Sweep line sorts events.
- Heap handles greedy with dynamic choices.

17. Practice Problems

Easy

- Assign Cookies, LeetCode, sort + greedy, Easy.
- Minimum Number of Arrows to Burst Balloons, LeetCode, interval greedy, Easy/Medium.

Medium

- Non-overlapping Intervals, LeetCode, sort by end, Medium.
- Boats to Save People, LeetCode, sort + two pointers, Medium.
- Queue Reconstruction by Height, LeetCode, sort + insert, Medium.

Hard

- Candy, LeetCode, greedy passes, Hard.
- Maximum Performance of a Team, LeetCode, sort + heap greedy, Hard.

18. Interview Explanation

Sorting creates a meaningful order, then greedy repeatedly chooses the locally safest option. I always explain the sorting key and why choosing it preserves an optimal solution.

19. Revision Notes

- Greedy needs proof.
- Sorting key is the main decision.
- Complexity usually $O(n \log n)$.
- Trap: equality boundaries.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Scheduling/pairing	sort + scan	$O(n \log n)$	choose safest local item	duplicates, equality

Frequency Arrays

1. Overview

Frequency arrays count occurrences of values when the value range is small or compressible.

2. Intuition

Instead of searching for how many times a value appears, keep a counter box for each possible value.

3. When to Use It

- Values lie in a small bounded range.
- Counting characters/digits.
- Trigger phrases: "frequency", "anagram", "count occurrences", "small constraints".

4. When Not to Use It

- Huge sparse values: use hash map or coordinate compression.
- Need sorted dynamic operations: use map/BST.
- Negative values unless shifted or compressed.

5. Core Concepts

- Direct indexing: `freq[value]++`.
- Character indexing: `freq[c - 'a']++`.
- Shift negatives: value `x` stored at `x + offset`.

6. Step-by-Step Algorithm

1. Determine value range.

2. Allocate frequency array.
3. Iterate through input and increment count.
4. Use counts for comparisons, reconstruction, or queries.

7. Dry Run

Array: `[1,3,1,2,3,1]`, max value 3.

value	frequency
0	0
1	3
2	1
3	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> buildFrequency(const vector<int>& a, int maxValue) {
    vector<int> freq(maxValue + 1, 0);
    for (int x : a) freq[x]++;
    return freq;
}

int main() {
    vector<int> a = {1,3,1,2,3,1};
    auto freq = buildFrequency(a, 3);
    cout << freq[1] << "\n"; // 3
}
```

pYTHON IMPLEMENTATION

```
def build_frequency(a, max_value):
    freq = [0] * (max_value + 1)
    for x in a:
        freq[x] += 1
    return freq
```

10. Code Explanation

The value itself is used as an index. This makes increment and lookup $O(1)$, but only if the range is safe to allocate.

11. Complexity Analysis

Operation	Time	Space
Build	$O(n)$	$O(\text{maxValue})$
Lookup	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Anagram	same character counts	frequency array	LeetCode 242
Counting sort	small integer range	freq then rebuild	GFG Counting Sort
Missing number	count presence	freq/boolean	LeetCode 448

13. Common Mistakes

- Allocating too large an array.
- Forgetting negative offset.
- Confusing frequency with prefix frequency.

14. Edge Cases

- Empty input.
- All same value.
- Max/min boundary value.
- Negative values.
- Large counts.

15. Variations

- Boolean presence array.
- Prefix frequency array for range count queries.
- 2D frequency for pairs/characters by position.

16. Related Algorithms/Data Structures

- Hash map handles sparse large keys.
- Coordinate compression converts large keys to small indices.
- Counting sort uses frequency arrays.

17. Practice Problems

Easy

- Valid Anagram, LeetCode, char frequency, Easy.
- Find All Numbers Disappeared in an Array, LeetCode, presence count, Easy.

Medium

- Sort Colors, LeetCode, counting or partitioning, Medium.
- Top K Frequent Elements, LeetCode, frequency + bucket/heap, Medium.

Hard

- Minimum Window Substring, LeetCode, frequency window, Hard.
- Count Good Triplets in an Array, LeetCode, freq/BIT, Hard.

18. Interview Explanation

When values come from a small range, I use an array as a direct counter. It is faster and simpler than a hash map because lookup is plain indexing.

19. Revision Notes

- Best for bounded values.
- $O(n + \text{range})$ if rebuilding.
- Shift or compress negatives.
- Trap: memory blowup.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Small-range counting	count/lookup	$O(n)/O(1)$	<code>freq[x]++</code>	negatives, huge range

Hashing with Arrays

1. Overview

Hashing with arrays uses hash maps/sets together with array scans to remember previously seen values, prefix sums, or states.

2. Intuition

When a future answer depends on whether something appeared before, store it. Then each index can ask the past in $O(1)$ average time.

3. When to Use It

- Need fast lookup of previous values/states.
- Unsorted array pair/subarray problems.
- Trigger phrases: "exists previous", "count subarrays", "distinct", "duplicates".

4. When Not to Use It

- Values are small: frequency array may be faster.

- Need ordered predecessor/successor: use balanced tree.
- Worst-case hashing concerns in adversarial CP: use custom hash or ordered map.

5. Core Concepts

- `unordered_map` for counts/indexes.
- Prefix sum state: if $\text{prefix}[j] - \text{prefix}[i] = k$, then need previous $\text{prefix} - k$.
- Store before/after depending on whether empty subarray is allowed.

6. Step-by-Step Algorithm

1. Choose state to store.
2. Initialize map for base case.
3. Scan array.
4. Query needed previous state.
5. Update answer.
6. Insert/update current state.

7. Dry Run

Count subarrays with sum 3, array `[1,2,1,2]`.

x	prefix	need	map before	added
1	1	-2	<code>{0:1}</code>	0
2	3	0	<code>{0:1,1:1}</code>	1
1	4	1	<code>{0:1,1:1,3:1}</code>	1
2	6	3	<code>{0:1,1:1,3:1,4:1}</code>	1

Answer: 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysWithSumK(const vector<int>& a, int k) {
    unordered_map<long long, int> seen;
    seen[0] = 1;
    long long prefix = 0, answer = 0;
    for (int x : a) {
        prefix += x;
        answer += seen[prefix - k];
        seen[prefix]++;
    }
    return answer;
}
```

```
int main() {
    vector<int> a = {1,2,1,2};
    cout << countSubarraysWithSumK(a, 3) << "\n";
}
```

pYTHON IMPLEMENTATION

```
from collections import defaultdict

def count_subarrays_with_sum_k(a, k):
    seen = defaultdict(int)
    seen[0] = 1
    prefix = ans = 0
    for x in a:
        prefix += x
        ans += seen[prefix - k]
        seen[prefix] += 1
    return ans
```

10. Code Explanation

For current prefix P , a previous prefix $P - k$ creates a subarray ending here with sum k . `seen[0] = 1` handles subarrays starting at index 0.

11. Complexity Analysis

Operation	Average Time	Space
Scan	$O(n)$	$O(n)$
Lookup/update	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Two Sum unsorted	complement lookup	hash map	LeetCode 1
Subarray sum K	previous prefix	prefix + map	LeetCode 560
Longest consecutive	set membership	hash set	LeetCode 128

13. Common Mistakes

- Forgetting base state.
- Updating map before querying when it creates invalid self-use.
- Using `map[key]` in C++ when accidental insertion matters.

14. Edge Cases

- Duplicate values.
- Negative values.
- Target zero.
- Empty array.
- Large prefix sums.

15. Variations

- Store first index for longest length.
- Store count for number of subarrays.
- Store last index for nearest distance.

16. Related Algorithms/Data Structures

- Frequency arrays for bounded keys.
- Prefix sum creates hashable states.
- Coordinate compression can replace hashing with arrays/BIT.

17. Practice Problems

Easy

- Two Sum, LeetCode, complement hash, Easy.
- Contains Duplicate, LeetCode, hash set, Easy.

Medium

- Subarray Sum Equals K, LeetCode, prefix hashing, Medium.
- Longest Consecutive Sequence, LeetCode, hash set, Medium.
- Continuous Subarray Sum, LeetCode, prefix modulo, Medium.

Hard

- Minimum Window Substring, LeetCode, hash counts, Hard.
- Count Subarrays With Median K, LeetCode, transformed prefix, Hard.

18. Interview Explanation

Hashing lets every array position query useful information from earlier positions in average $O(1)$. For prefix problems, I store prefix sums or counts of prefix states.

19. Revision Notes

- Choose state carefully.
- Initialize base case.
- Query before update for subarrays ending here.
- Trap: duplicates require counts.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Previous-state lookup	query/update map	$O(n)$ avg	store complement/prefix	duplicates, zero, negatives

In-place Modification

1. Overview

In-place modification changes an array without using significant extra space, often with slow/fast pointers or value marking.

2. Intuition

Reuse the original array as workspace. Keep a write pointer for the next valid position, or encode information inside values.

3. When to Use It

- Space constraint $O(1)$.
- Remove duplicates/elements.
- Rearrange values.
- Trigger phrases: "modify in-place", "constant extra space", "return new length".

4. When Not to Use It

- Input must remain unchanged.
- Encoding would overflow or conflict with allowed values.
- Simpler extra-space solution is acceptable and clearer.

5. Core Concepts

- Write pointer: position where next kept element goes.
- Swap partitioning: move categories to correct sides.
- Marking by sign/index: use values to store visited info.

6. Step-by-Step Algorithm

1. Choose invariant for processed prefix.
2. Maintain a `write` index.
3. Scan each element.
4. If element should remain, assign `a[write] = a[i]` and increment `write`.
5. Return `write` or modified array.

7. Dry Run

Remove value 3 from `[3, 2, 2, 3]`.

i	a[i]	write	action	array
0	3	0	skip	[3,2,2,3]
1	2	0	write 2	[2,2,2,3]
2	2	1	write 2	[2,2,2,3]
3	3	2	skip	[2,2,2,3]

New length: 2.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int removeValue(vector<int>& a, int value) {
    int write = 0;
    for (int x : a) {
        if (x != value) {
            a[write] = x;
            write++;
        }
    }
    return write;
}

int main() {
    vector<int> a = {3,2,2,3};
    int len = removeValue(a, 3);
    for (int i = 0; i < len; i++) cout << a[i] << " ";
}
```

pYTHON IMPLEMENTATION

```
def remove_value(a, value):
    write = 0
    for x in a:
        if x != value:
            a[write] = x
            write += 1
    return write
```

10. Code Explanation

The prefix `[0, write)` always contains valid kept elements. Elements after `write` may be garbage and are ignored by the returned length.

11. Complexity Analysis

Operation	Time	Extra Space
Scan and write	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Remove elements	return length	write pointer	LeetCode 27
Remove duplicates	sorted array	slow pointer	LeetCode 26
Mark missing	values 1..n	sign marking	LeetCode 448

13. Common Mistakes

- Reading modified values incorrectly.
- Returning array size instead of logical length.
- Sign marking without using `abs`.

14. Edge Cases

- All removed.
- None removed.
- Empty array.
- Duplicates.
- Values at boundary for index marking.

15. Variations

- Cyclic sort for values `1..n`.
- Dutch national flag.
- Negative marking for missing/duplicate detection.

16. Related Algorithms/Data Structures

- Two pointers for compaction.
- Partitioning for category rearrangement.
- Frequency arrays if extra space is allowed.

17. Practice Problems

Easy

- Remove Element, LeetCode, write pointer, Easy.
- Move Zeroes, LeetCode, in-place compaction, Easy.

Medium

- Sort Colors, LeetCode, in-place partitioning, Medium.
- Find All Duplicates in an Array, LeetCode, sign marking, Medium.

Hard

- First Missing Positive, LeetCode, index placement, Hard.
- Trapping Rain Water, LeetCode, in-place/two pointers, Hard.

18. Interview Explanation

I keep an invariant over the processed part of the array and use a write pointer or swaps to place elements directly into their final logical positions.

19. Revision Notes

- Define processed prefix invariant.
- Return logical length.
- $O(1)$ extra space.
- Trap: input mutation may be disallowed.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Space-limited rearrange	scan/write/swap	$O(n)$	maintain invariant	all removed, mutated input

Rotate Array

1. Overview

Array rotation shifts elements cyclically by k positions.

2. Intuition

Rotating right by k means the last k elements move to the front while preserving order. Reversal solves it cleanly: reverse all, then reverse the two parts.

3. When to Use It

- Cyclic shifts.
- Trigger phrases: "rotate by k ", "cyclically shift", "wrap around".
- In-place array manipulation questions.

4. When Not to Use It

- If many rotations and queries are needed, store an offset instead of physically rotating.
- If order does not matter, simpler swaps may suffice.

- Linked lists need different pointer logic.

5. Core Concepts

- Normalize: $k \% n$.
- Right rotation by k : reverse whole, reverse $[0, k-1]$, reverse $[k, n-1]$.
- Left rotation by k equals right rotation by $n-k$.

6. Step-by-Step Algorithm

1. If $n == 0$, return.
2. Set $k \% n$.
3. Reverse the whole array.
4. Reverse first k elements.
5. Reverse remaining $n - k$ elements.

7. Dry Run

Array $[1, 2, 3, 4, 5, 6, 7]$, right rotate 3.

Step	Array
original	$[1, 2, 3, 4, 5, 6, 7]$
reverse all	$[7, 6, 5, 4, 3, 2, 1]$
reverse first 3	$[5, 6, 7, 4, 3, 2, 1]$
reverse rest	$[5, 6, 7, 1, 2, 3, 4]$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void rotateRight(vector<int>& a, int k) {
    int n = (int)a.size();
    if (n == 0) return;
    k %= n;
    reverse(a.begin(), a.end());
    reverse(a.begin(), a.begin() + k);
    reverse(a.begin() + k, a.end());
}

int main() {
    vector<int> a = {1, 2, 3, 4, 5, 6, 7};
    rotateRight(a, 3);
    for (int x : a) cout << x << " ";
}
```

pYTHON IMPLEMENTATION

```
def rotate_right(a, k):  
    n = len(a)  
    if n == 0:  
        return  
    k %= n  
    a[:] = a[-k:] + a[:-k] if k else a
```

10. Code Explanation

Modulo handles rotations larger than array length. Reversing all brings the desired tail to the front, but reversed internally; reversing each part restores internal order.

11. Complexity Analysis

Operation	Time	Extra Space
Reverse method	$O(n)$	$O(1)$ in C++
Slicing Python version	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Rotate by k	cyclic shift	reverse method	LeetCode 189
Search rotated	sorted then rotated	binary search	LeetCode 33
Circular index	wrap around	modulo	CP circular arrays

13. Common Mistakes

- Not doing $k \% n$.
- Division by zero when $n = 0$.
- Confusing left and right rotation.

14. Edge Cases

- Empty array.
- $k = 0$.
- $k > n$.
- $n = 1$.
- Duplicate values.

15. Variations

- Juggling algorithm using gcd cycles.

- Block swap rotation.
- Lazy rotation using offset.

16. Related Algorithms/Data Structures

- Reversal algorithm.
- Circular arrays use modulo without physical rotation.
- Binary search on rotated arrays.

17. Practice Problems

Easy

- Rotate Array, LeetCode, reverse method, Easy/Medium.
- Left Rotate an Array, GFG, reversal, Easy.

Medium

- Search in Rotated Sorted Array, LeetCode, binary search, Medium.
- Find Minimum in Rotated Sorted Array, LeetCode, binary search, Medium.

Hard

- Search in Rotated Sorted Array II, LeetCode, duplicates, Hard-ish.
- Rotate Image, LeetCode, matrix rotation, Hard-ish.

18. Interview Explanation

I normalize k , then use three reversals to rotate in-place. The first reversal moves the tail to the front, and the next two restore order inside both parts.

19. Revision Notes

- $k \% n$.
- Right rotate: reverse all, first k , rest.
- $O(n)$, $O(1)$.
- Trap: empty array.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Cyclic shift	rotate	$O(n)$	3 reversals	$n=0$, $k>n$

Merge Intervals

1. Overview

Merge intervals combines overlapping intervals into disjoint intervals.

2. Intuition

Sort intervals by start. Then only the current interval can overlap with the last merged interval. If it overlaps, extend the end; otherwise start a new merged interval.

3. When to Use It

- Overlapping time ranges.
- Calendar/free time problems.
- Trigger phrases: "merge overlapping", "covered ranges", "interval union".

4. When Not to Use It

- Need maximum overlap count: use sweep line.
- Need online insertions/deletions: use balanced tree.
- Intervals are points/events only: sort events.

5. Core Concepts

- Sort by start coordinate.
- Overlap condition: `next.start <= current.end` for closed intervals.
- Merge end: `current.end = max(current.end, next.end)`.

6. Step-by-Step Algorithm

1. Sort intervals by start.
2. Initialize answer with first interval.
3. For each next interval, compare with answer's last interval.
4. If overlapping, update last end.
5. Otherwise append it.

7. Dry Run

Intervals: `[[1,3],[2,6],[8,10],[15,18]]`.

interval	last	action
[1,3]	none	add
[2,6]	[1,3]	merge to [1,6]
[8,10]	[1,6]	add
[15,18]	[8,10]	add

Answer: `[[1,6],[8,10],[15,18]]`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
    if (intervals.empty()) return {};
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> merged;
    for (auto interval : intervals) {
        if (merged.empty() || interval[0] > merged.back()[1]) {
            merged.push_back(interval);
        } else {
            merged.back()[1] = max(merged.back()[1], interval[1]);
        }
    }
    return merged;
}

int main() {
    vector<vector<int>> intervals = {{1,3},{2,6},{8,10},{15,18}};
    for (auto v : mergeIntervals(intervals)) cout << v[0] << "," << v[1] << "\n";
}
```

pYTHON IMPLEMENTATION

```
def merge_intervals(intervals):
    intervals.sort()
    merged = []
    for start, end in intervals:
        if not merged or start > merged[-1][1]:
            merged.append([start, end])
        else:
            merged[-1][1] = max(merged[-1][1], end)
    return merged
```

10. Code Explanation

Sorting guarantees intervals that can overlap are adjacent in processing order. The answer's last interval represents the current union block.

11. Complexity Analysis

Operation	Time	Space
Sort	$O(n \log n)$	$O(1)$ to $O(n)$
Merge scan	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Merge ranges	interval union	sort + merge	LeetCode 56
Insert interval	add then merge	linear merge	LeetCode 57
Erase overlap	remove minimum	sort by end greedy	LeetCode 435

13. Common Mistakes

- Wrong overlap condition for open vs closed intervals.
- Forgetting empty input.
- Sorting by end when merge requires start order.

14. Edge Cases

- Empty intervals.
- One interval.
- Touching intervals.
- Fully nested intervals.
- Same start different end.

15. Variations

- Insert interval into sorted disjoint intervals.
- Meeting rooms count using sweep/heap.
- Interval intersection between two lists.

16. Related Algorithms/Data Structures

- Sweep line counts overlaps.
- Sorting + greedy handles interval selection.
- Difference array handles coverage on small coordinate range.

17. Practice Problems

Easy

- Merge Intervals, LeetCode, sort + merge, Easy/Medium.
- Meeting Rooms, LeetCode/LintCode, overlap check, Easy.

Medium

- Insert Interval, LeetCode, interval merging, Medium.
- Interval List Intersections, LeetCode, two pointers, Medium.

Hard

- Employee Free Time, LeetCode, merge schedules, Hard.
- Amount of New Area Painted Each Day, LeetCode, intervals/DSU, Hard.

18. Interview Explanation

After sorting by start, any overlap with the current block must appear next. I keep the last merged interval and either extend it or start a new one.

19. Revision Notes

- Sort by start.
- Overlap if `start <= lastEnd`.
- $O(n \log n)$.
- Trap: closed vs open interval boundaries.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Interval union	sort/merge	$O(n \log n)$	compare with last merged	touching, nested, empty

Subarray Sum Problems

1. Overview

Subarray sum problems ask about contiguous segments with a target, maximum, minimum, divisibility, or count property.

2. Intuition

A subarray sum is the difference between two prefix sums. Once you see that, many problems become "find previous prefix state that makes current state valid."

3. When to Use It

- Contiguous subarray with sum condition.
- Negative values are present.
- Trigger phrases: "subarray sum equals K", "sum divisible by K", "count subarrays".

4. When Not to Use It

- Non-negative array with longest/shortest bounded sum: sliding window may be simpler.
- Non-contiguous subsequence: use DP/greedy.
- Need dynamic updates: Fenwick/segment tree.

5. Core Concepts

- Sum `[l, r] = pref[r + 1] - pref[l]`.
- Count target `k`: count previous `pref - k`.
- Divisible by `k`: same prefix remainder.

6. Step-by-Step Algorithm

1. Initialize prefix sum and hash map.
2. Add base state.
3. Scan each element and update prefix.
4. Convert problem condition into needed previous prefix.
5. Update answer and map.

7. Dry Run

Array `[4,5,0,-2,-3,1]`, count subarrays divisible by 5.

x	prefix mod 5	previous count	ans
4	4	0	0
5	4	1	1
0	4	2	3
-2	2	0	3
-3	4	3	6
1	0	1	7

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysDivisibleByK(const vector<int>& a, int k) {
    vector<long long> count(k, 0);
    count[0] = 1;
    long long prefix = 0, answer = 0;
    for (int x : a) {
        prefix = (prefix + x) % k;
        if (prefix < 0) prefix += k;
        answer += count[prefix];
        count[prefix]++;
    }
    return answer;
}

int main() {
    vector<int> a = {4,5,0,-2,-3,1};
    cout << countSubarraysDivisibleByK(a, 5) << "\n";
}
```

pYTHON IMPLEMENTATION

```
def count_subarrays_divisible_by_k(a, k):  
    count = [0] * k  
    count[0] = 1  
    prefix = ans = 0  
    for x in a:  
        prefix = (prefix + x) % k  
        ans += count[prefix]  
        count[prefix] += 1  
    return ans
```

10. Code Explanation

If two prefix sums have the same remainder modulo k , their difference is divisible by k . Each current prefix can pair with all previous equal remainders.

11. Complexity Analysis

Operation	Time	Space
Prefix scan	$O(n)$	$O(k)$ or $O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Exact sum	sum equals K	prefix + map	LeetCode 560
Divisible sum	sum % $K == 0$	prefix remainder counts	LeetCode 974
Binary exact count	exactly K ones	prefix count/map	LeetCode 930

13. Common Mistakes

- Using sliding window with negative numbers.
- Not normalizing negative modulo in C++.
- Forgetting base prefix 0 .

14. Edge Cases

- Target 0 .
- Negative numbers.
- All zeros.
- Large sums.
- $k = 1$.

15. Variations

- Longest subarray with sum K : store first prefix index.
- Count subarrays with odd count K : prefix parity/count.

- Range sum count: prefix + merge sort/BIT.

16. Related Algorithms/Data Structures

- Prefix sum is the base.
- Hash map stores previous states.
- Sliding window works for non-negative monotonic sums.

17. Practice Problems

Easy

- Running Sum of 1d Array, LeetCode, prefix basics, Easy.
- Find Pivot Index, LeetCode, subarray balance, Easy.

Medium

- Subarray Sum Equals K, LeetCode, prefix map, Medium.
- Subarray Sums Divisible by K, LeetCode, prefix mod, Medium.
- Binary Subarrays With Sum, LeetCode, prefix/sliding, Medium.

Hard

- Count of Range Sum, LeetCode, prefix + merge sort, Hard.
- Shortest Subarray with Sum at Least K, LeetCode, prefix + deque, Hard.

18. Interview Explanation

I convert subarray sums into differences of prefix sums. Then I store previous prefix states so each endpoint can count or find valid starts efficiently.

19. Revision Notes

- Subarray sum = prefix difference.
- Store counts for counting.
- Store first index for longest.
- Trap: negative values and modulo.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Contiguous sum conditions	prefix + lookup	$O(n)$	previous prefix state	negatives, zero, modulo

Maximum/Minimum Subarray

1. Overview

Maximum/minimum subarray problems find the best contiguous segment by sum, product, or transformed score.

2. Intuition

For sums, decide whether to extend or restart. For minimum, mirror the logic. For product, track both max and min because a negative flips signs.

3. When to Use It

- Best/worst contiguous segment.
- Trigger phrases: "maximum subarray", "minimum subarray", "contiguous".
- Difference arrays in stock-profit style problems.

4. When Not to Use It

- Fixed length: use sliding window.
- Non-contiguous: use subsequence DP.
- Need arbitrary range max subarray queries: segment tree with states.

5. Core Concepts

- Max sum: Kadane.
- Min sum: replace max with min.
- Product: keep **maxEnding** and **minEnding**.

6. Step-by-Step Algorithm

1. Initialize current and best with first element.
2. For each element, decide extend/restart.
3. Update best.
4. For product, swap max/min when multiplying by negative.

7. Dry Run

Minimum sum for **[3, -4, 2, -3, -1, 7]**.

x	currentMin	bestMin
3	3	3
-4	-4	-4
2	-2	-4
-3	-5	-5
-1	-6	-6
7	1	-6

Answer: **-6**.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

pair<long long, long long> maxAndMinSubarraySum(const vector<int>& a) {
    long long curMax = a[0], bestMax = a[0];
    long long curMin = a[0], bestMin = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        curMax = max<long long>(a[i], curMax + a[i]);
        bestMax = max(bestMax, curMax);
        curMin = min<long long>(a[i], curMin + a[i]);
        bestMin = min(bestMin, curMin);
    }
    return {bestMax, bestMin};
}

int main() {
    vector<int> a = {3, -4, 2, -3, -1, 7};
    auto [mx, mn] = maxAndMinSubarraySum(a);
    cout << mx << " " << mn << "\n";
}
```

pYTHON IMPLEMENTATION

```
def max_and_min_subarray_sum(a):
    cur_max = best_max = a[0]
    cur_min = best_min = a[0]
    for x in a[1:]:
        cur_max = max(x, cur_max + x)
        best_max = max(best_max, cur_max)
        cur_min = min(x, cur_min + x)
        best_min = min(best_min, cur_min)
    return best_max, best_min
```

10. Code Explanation

`curMax` and `curMin` represent best/worst subarray ending at current index. The answer is updated after each endpoint.

11. Complexity Analysis

Operation	Time	Space
Kadane-style scan	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Max sum	best contiguous sum	Kadane	LeetCode 53
Min sum	worst contiguous sum	min Kadane	circular subarray
Max product	sign flips	max/min product DP	LeetCode 152

13. Common Mistakes

- Not handling all-negative max sum.
- Product problems ignoring negative minimum.
- Confusing subarray with subsequence.

14. Edge Cases

- Single element.
- All negative.
- All positive.
- Zeros in product.
- Large values/overflow.

15. Variations

- Maximum product subarray.
- Maximum circular subarray.
- Maximum subarray with one deletion.

16. Related Algorithms/Data Structures

- Kadane is the core.
- Prefix min/max can solve max subarray by prefix differences.
- Segment tree can answer range max subarray queries.

17. Practice Problems

Easy

- Maximum Subarray, LeetCode, Kadane, Easy/Medium.
- Maximum Difference Between Increasing Elements, LeetCode, prefix min, Easy.

Medium

- Maximum Product Subarray, LeetCode, max/min product, Medium.
- Maximum Sum Circular Subarray, LeetCode, max/min Kadane, Medium.

Hard

- Maximum Subarray Sum with One Deletion, LeetCode, DP, Hard.
- Shortest Subarray with Sum at Least K, LeetCode, prefix deque, Hard.

18. Interview Explanation

For best contiguous sums, I track the best subarray ending at each position. The current element either extends the previous segment or starts a new one.

19. Revision Notes

- Max: $\max(x, \text{cur}+x)$.
- Min: $\min(x, \text{cur}+x)$.
- Product needs both max and min.
- Trap: initialization with 0.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Best/worst segment	scan DP	$O(n)$	extend/restart	all negative, zero product

Product Except Self

1. Overview

Product except self computes for every index the product of all other elements without using division.

2. Intuition

The answer at i is product of everything left of i times product of everything right of i .

3. When to Use It

- Need product excluding current index.
- Division forbidden or zero values exist.
- Trigger phrases: "except self", "without division", "all other elements".

4. When Not to Use It

- Product can overflow beyond available type.
- Need dynamic updates: use segment tree/product Fenwick with zero handling.
- Division is allowed and zero cases are simple enough.

5. Core Concepts

- Prefix product before index.
- Suffix product after index.
- $O(1)$ extra space excluding output by storing prefix in output and multiplying suffix on the fly.

6. Step-by-Step Algorithm

1. Initialize answer with ones.

2. Scan left to right storing product before current index.
3. Scan right to left multiplying product after current index.
4. Return answer.

7. Dry Run

Array `[1,2,3,4]`.

Left pass ans: `[1,1,2,6]`.

Right pass:

i	suffix	ans[i]
3	1	6
2	4	8
1	12	12
0	24	24

Answer: `[24,12,8,6]`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> productExceptSelf(const vector<int>& a) {
    int n = (int)a.size();
    vector<long long> ans(n, 1);
    long long leftProduct = 1;
    for (int i = 0; i < n; i++) {
        ans[i] = leftProduct;
        leftProduct *= a[i];
    }
    long long rightProduct = 1;
    for (int i = n - 1; i >= 0; i--) {
        ans[i] *= rightProduct;
        rightProduct *= a[i];
    }
    return ans;
}

int main() {
    vector<int> a = {1,2,3,4};
    for (long long x : productExceptSelf(a)) cout << x << " ";
}
```

pYTHON IMPLEMENTATION

```
def product_except_self(a):
    n = len(a)
    ans = [1] * n
    left = 1
    for i, x in enumerate(a):
        ans[i] = left
        left *= x
    right = 1
    for i in range(n - 1, -1, -1):
        ans[i] *= right
        right *= a[i]
    return ans
```

10. Code Explanation

The first pass stores left products. The second pass multiplies each answer by the right product currently available before including `a[i]`.

11. Complexity Analysis

Operation	Time	Extra Space
Two passes	$O(n)$	$O(1)$ excluding output

12. Common Patterns

Pattern	Identify	Approach	Examples
Except self	product of others	prefix/suffix product	LeetCode 238
Left/right contribution	combine sides	prefix + suffix	Trapping Rain Water
Zero-safe product	no division	two passes	OA array products

13. Common Mistakes

- Multiplying current element into prefix too early.
- Using division and failing on zeros.
- Overflow.

14. Edge Cases

- One zero.
- Multiple zeros.
- Single element.
- Negative values.
- Large products.

15. Variations

- Product except self with modulo.
- Sum except self: total sum minus current.
- Dynamic product queries with segment tree.

16. Related Algorithms/Data Structures

- Prefix/suffix arrays.
- Segment tree for dynamic products.
- Modular inverse if division under prime modulo is valid and no zero.

17. Practice Problems

Easy

- Left and Right Sum Differences, LeetCode, side products/sums idea, Easy.
- Product Array Puzzle, GFG, prefix/suffix, Easy.

Medium

- Product of Array Except Self, LeetCode, prefix/suffix, Medium.
- Construct Product Matrix, LeetCode, prefix/suffix modulo, Medium.

Hard

- Maximum Product Subarray, LeetCode, product DP, Hard-ish.
- Product of the Last K Numbers, LeetCode, prefix product with zeros, Hard-ish.

18. Interview Explanation

For each index, I multiply the product of all elements before it and after it. Two passes let me do that without division and with constant extra space besides output.

19. Revision Notes

- Answer starts as left products.
- Second pass multiplies suffix.
- Handles zeros naturally.
- Trap: overflow.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Product excluding index	two passes	$O(n)$	left then right product	zeros, overflow

Partitioning

1. Overview

Partitioning rearranges an array so elements satisfying a condition are placed before/after others.

2. Intuition

Maintain regions: processed small values, processed middle values, unknown values, and processed large values. Move elements into their correct region by swaps.

3. When to Use It

- Group elements by condition/category.
- Quickselect/quicksort.
- Trigger phrases: "partition", "sort colors", "less than pivot", "odd even".

4. When Not to Use It

- Need stable order: basic swap partition is not stable.
- Many categories: counting sort may be simpler.
- Need fully sorted order: partition is only partial.

5. Core Concepts

- Lomuto partition: one boundary for `< pivot`.
- Hoare partition: two pointers moving inward.
- Dutch national flag: three categories.

6. Step-by-Step Algorithm

1. Set `low = 0`, `mid = 0`, `high = n - 1`.
2. If `a[mid] == 0`, swap with `low`, increment both.
3. If `a[mid] == 1`, increment `mid`.
4. If `a[mid] == 2`, swap with `high`, decrement `high`.
5. Continue while `mid <= high`.

7. Dry Run

Sort colors `[2,0,2,1,1,0]`.

low	mid	high	action	array
0	0	5	swap 2 with high	<code>[0,0,2,1,1,2]</code>
0	0	4	swap 0 with low	<code>[0,0,2,1,1,2]</code>
1	1	4	swap 0 with low	<code>[0,0,2,1,1,2]</code>
2	2	4	swap 2 with high	<code>[0,0,1,1,2,2]</code>
2	2	3	mid++	<code>[0,0,1,1,2,2]</code>
2	3	3	mid++	done

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

void sortColors(vector<int>& a) {
    int low = 0, mid = 0, high = (int)a.size() - 1;
    while (mid <= high) {
        if (a[mid] == 0) swap(a[low++], a[mid++]);
        else if (a[mid] == 1) mid++;
        else swap(a[mid], a[high--]);
    }
}

int main() {
    vector<int> a = {2,0,2,1,1,0};
    sortColors(a);
    for (int x : a) cout << x << " ";
}

```

pYTHON IMPLEMENTATION

```

def sort_colors(a):
    low = mid = 0
    high = len(a) - 1
    while mid <= high:
        if a[mid] == 0:
            a[low], a[mid] = a[mid], a[low]
            low += 1
            mid += 1
        elif a[mid] == 1:
            mid += 1
        else:
            a[mid], a[high] = a[high], a[mid]
            high -= 1

```

10. Code Explanation

$[0, low)$ contains zeros, $[low, mid)$ contains ones, $[mid, high]$ is unknown, and $(high, n)$ contains twos. After swapping with $high$, mid is not incremented because the incoming value is unknown.

11. Complexity Analysis

Operation	Time	Extra Space
Partition scan	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Binary partition	true/false grouping	two pointers	Move Zeroes
Three-way partition	0/1/2 or < = > pivot	DNF	LeetCode 75
Selection	kth element	quickselect partition	LeetCode 215

13. Common Mistakes

- Incrementing `mid` after swapping with `high`.
- Assuming partition is stable.
- Bad pivot causing quicksort worst case.

14. Edge Cases

- Empty array.
- All one category.
- Already partitioned.
- Reverse grouped.
- Duplicates around pivot.

15. Variations

- Stable partition using extra space.
- Hoare partition for quicksort.
- Quickselect for kth largest/smallest.

16. Related Algorithms/Data Structures

- In-place modification uses similar invariants.
- Sorting can be built from partitioning.
- Two pointers drive partition boundaries.

17. Practice Problems

Easy

- Move Zeroes, LeetCode, binary partition, Easy.
- Sort Array by Parity, LeetCode, two-way partition, Easy.

Medium

- Sort Colors, LeetCode, Dutch flag, Medium.
- Kth Largest Element in an Array, LeetCode, quickselect, Medium.

Hard

- Wiggle Sort II, LeetCode, partition + median, Hard.
- First Missing Positive, LeetCode, index partitioning, Hard.

18. Interview Explanation

Partitioning keeps clear regions in the array and moves each encountered value into its region. The invariant makes the one-pass in-place logic easy to prove.

19. Revision Notes

- Define regions.
- Do not increment `mid` after high swap in DNF.
- $O(n)$, $O(1)$.
- Trap: stability.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Group by condition	swaps	$O(n)$	maintain regions	all same, stability

Prefix Minimum/Maximum

1. Overview

Prefix minimum/maximum stores the best value seen so far up to each index.

2. Intuition

At every position, remember the cheapest/biggest thing before it. This lets you answer "best partner on the left" immediately.

3. When to Use It

- Need best previous value for each index.
- Stock buy/sell.
- Trigger phrases: "minimum before", "maximum so far", "best left side".

4. When Not to Use It

- Need dynamic updates.
- Need arbitrary range min/max queries: sparse table or segment tree.
- Need future values: use suffix min/max.

5. Core Concepts

- `prefMin[i] = min(a[0..i])`.
- `prefMax[i] = max(a[0..i])`.
- Often can be stored in one variable.

6. Step-by-Step Algorithm

1. Initialize best value with first element.
2. Scan left to right.
3. Use current best to calculate answer involving current index.
4. Update best with current element.

7. Dry Run

Max stock profit prices [7,1,5,3,6,4].

price	minSoFar	profit	best
7	7	0	0
1	1	0	0
5	1	4	4
3	1	2	4
6	1	5	5
4	1	3	5

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int maxProfitOneTransaction(const vector<int>& prices) {
    int minSoFar = INT_MAX, best = 0;
    for (int price : prices) {
        minSoFar = min(minSoFar, price);
        best = max(best, price - minSoFar);
    }
    return best;
}

int main() {
    vector<int> prices = {7,1,5,3,6,4};
    cout << maxProfitOneTransaction(prices) << "\n";
}
```

pYTHON IMPLEMENTATION

```
def max_profit_one_transaction(prices):
    min_so_far = float("inf")
    best = 0
    for price in prices:
        min_so_far = min(min_so_far, price)
```

```
best = max(best, price - min_so_far)
return best
```

10. Code Explanation

For each selling price, the best buy price must be the minimum seen earlier or at the current index. Updating `best` with `price - minSoFar` tests that sell day.

11. Complexity Analysis

Operation	Time	Space
One-pass variable	$O(n)$	$O(1)$
Full prefix array	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Stock profit	sell after buy	prefix min	LeetCode 121
Max difference	$a[j] - a[i], i < j$	prefix min	GFG max difference
Rain water	left/right max	prefix/suffix max	LeetCode 42

13. Common Mistakes

- Updating answer before/after best incorrectly depending on strict previous requirement.
- Using prefix min when future min is needed.
- Not handling decreasing arrays.

14. Edge Cases

- Single element.
- Strictly decreasing.
- Strictly increasing.
- Equal values.
- Negative values.

15. Variations

- Suffix min/max.
- Prefix best pair.
- Prefix max plus suffix max for two transactions.

16. Related Algorithms/Data Structures

- Prefix sum stores cumulative totals.
- Kadane on differences solves stock profit too.

- Sparse table handles arbitrary static range min/max.

17. Practice Problems

Easy

- Best Time to Buy and Sell Stock, LeetCode, prefix min, Easy.
- Maximum Difference Between Increasing Elements, LeetCode, prefix min, Easy.

Medium

- Trapping Rain Water, LeetCode, prefix/suffix max, Medium/Hard.
- Maximum Score Sightseeing Pair, LeetCode, prefix best expression, Medium.

Hard

- Best Time to Buy and Sell Stock III, LeetCode, prefix/suffix DP, Hard.
- Maximum Sum of Three Non-Overlapping Subarrays, LeetCode, prefix best, Hard.

18. Interview Explanation

Prefix min/max keeps the best value on the left while scanning. This turns problems that compare current value with a previous best into a simple $O(n)$ pass.

19. Revision Notes

- Best-so-far variable is often enough.
- Choose min or max based on formula.
- $O(n)$, $O(1)$.
- Trap: whether current index may be included.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Best previous value	scan	$O(n)$	<code>best=min/max(best,a[i])</code>	strict ordering, monotonic arrays

Contribution Technique

1. Overview

Contribution technique computes a total by summing how much each element contributes instead of enumerating all subarrays/pairs.

2. Intuition

Flip the question. Instead of asking every subarray what it contains, ask each element: "In how many valid structures do you appear, and with what weight?"

3. When to Use It

- Sum over all subarrays/pairs/subsequences.
- Trigger phrases: "sum of all subarrays", "total contribution", "all pairs".
- Need $O(n)$ or $O(n \log n)$ instead of $O(n^2)$.

4. When Not to Use It

- Need list of all subarrays, not aggregate.
- Contribution count is hard due to complex dependencies.
- Small n where brute force is acceptable.

5. Core Concepts

- Element at index i appears in $(i + 1) * (n - i)$ subarrays.
- For min/max over subarrays, use monotonic stack to find span.
- For pair contribution after sorting: difference multiplied by count.

6. Step-by-Step Algorithm

1. Decide what each element contributes.
2. Count how many structures include it.
3. Multiply value by count/weight.
4. Sum all contributions.

7. Dry Run

Sum of all subarray sums for $[1, 2, 3]$.

i	value	left choices	right choices	contribution
0	1	1	3	3
1	2	2	2	8
2	3	3	1	9

Total: 20.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long sumOfAllSubarraySums(const vector<int>& a) {
    int n = (int)a.size();
    long long answer = 0;
    for (int i = 0; i < n; i++) {
        long long count = 1LL * (i + 1) * (n - i);
        answer += count * a[i];
    }
}
```

```
        return answer;
    }

    int main() {
        vector<int> a = {1,2,3};
        cout << sumOfAllSubarraySums(a) << "\n";
    }
```

pYTHON IMPLEMENTATION

```
def sum_of_all_subarray_sums(a):
    n = len(a)
    ans = 0
    for i, x in enumerate(a):
        ans += x * (i + 1) * (n - i)
    return ans
```

10. Code Explanation

An element at i can choose any start from $0..i$ and any end from $i..n-1$. Multiplying those choices gives the number of subarrays containing it.

11. Complexity Analysis

Operation	Time	Space
Simple contribution	$O(n)$	$O(1)$
With monotonic stack	$O(n)$	$O(n)$
With sorting	$O(n \log n)$	$O(1)$ to $O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Sum all subarray sums	aggregate over all subarrays	index contribution	GFG sum of subarray sums
Sum subarray minimums	each element as minimum	monotonic stack	LeetCode 907
Pair distances	all pairs after sorting	contribution by order	LeetCode 1685

13. Common Mistakes

- Off-by-one in left/right counts.
- Wrong tie handling in monotonic stacks.

- Overflow without `long long`.

14. Edge Cases

- Duplicates.
- Negative values.
- Single element.
- Large n and values.
- Modulo requirement.

15. Variations

- Contribution to subarray minimum/maximum.
- Pair contribution after sorting.
- Bit contribution across all XOR/OR sums.

16. Related Algorithms/Data Structures

- Monotonic stack finds dominance ranges.
- Sorting enables pair contribution.
- Prefix sum can compute some aggregate sums too.

17. Practice Problems

Easy

- Sum of All Subarray Sums, GFG, index contribution, Easy.
- Sum Absolute Differences in a Sorted Array, LeetCode, sorted contribution, Easy/Medium.

Medium

- Sum of Subarray Minimums, LeetCode, monotonic contribution, Medium.
- Total Appeal of A String, LeetCode, last occurrence contribution, Medium.

Hard

- Sum of Subsequence Widths, LeetCode, sorted contribution, Hard.
- Subarray Ranges, LeetCode, min/max contribution, Hard-ish.

18. Interview Explanation

Contribution technique avoids enumerating all structures. I calculate how many times each element affects the final answer and add those weighted contributions.

19. Revision Notes

- Flip from structures to elements.
- Basic count: $(i+1)*(n-i)$.
- Use monotonic stack for min/max.
- Trap: duplicate tie rules.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Aggregate over many subarrays/pairs	count contribution	$O(n)$ - $O(n \log n)$	value * occurrences	duplicates, overflow

Coordinate Compression

1. Overview

Coordinate compression maps large or sparse values to small indices while preserving order.

2. Intuition

If only relative order matters, replace actual values like 10^9 with ranks like 0, 1, 2. This makes array-based structures possible.

3. When to Use It

- Values are large but number of distinct values is manageable.
- Need Fenwick/segment tree/frequency array over values.
- Trigger phrases: "large coordinates", "values up to $1e9$ ", "rank".

4. When Not to Use It

- Actual distances matter, unless storing gaps separately.
- Values are already small.
- Online unseen values arrive and cannot be preprocessed.

5. Core Concepts

- Sort unique values.
- Compressed index = lower_bound position.
- Order is preserved: $x < y$ implies $\text{comp}[x] < \text{comp}[y]$.

6. Step-by-Step Algorithm

1. Copy all values that need compression.
2. Sort them.
3. Remove duplicates.
4. For each original value, binary search its index in unique list.

7. Dry Run

Values: [100, -5, 100, 7].

Sorted unique: [-5, 7, 100].

value	compressed
100	2
-5	0
100	2
7	1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> compressValues(const vector<int>& a) {
    vector<int> values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    vector<int> compressed;
    for (int x : a) {
        int id = lower_bound(values.begin(), values.end(), x) - values.begin();
        compressed.push_back(id);
    }
    return compressed;
}

int main() {
    vector<int> a = {100, -5, 100, 7};
    for (int x : compressValues(a)) cout << x << " ";
}
```

pYTHON IMPLEMENTATION

```
from bisect import bisect_left

def compress_values(a):
    values = sorted(set(a))
    return [bisect_left(values, x) for x in a]
```

10. Code Explanation

The sorted unique list defines ranks. `lower_bound` finds each value's rank, preserving comparisons while shrinking memory requirements.

11. Complexity Analysis

Operation	Time	Space
Sort unique	$O(n \log n)$	$O(n)$
Compress all	$O(n \log n)$	$O(n)$
With hash rank map	$O(n)$ after sorting	$O(n)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Inversions	large values	compress + BIT	CSES Inversions
Range frequency	sparse coords	compress + arrays	CP queries
Sweep line	large endpoints	compress coordinates	Rectangle union basics

13. Common Mistakes

- Compressing only array values but forgetting query endpoints.
- Using compressed index as actual distance.
- Forgetting duplicates.

14. Edge Cases

- Negative values.
- Duplicates.
- One distinct value.
- Large coordinates.
- Query coordinates not in original array.

15. Variations

- 1-based compression for Fenwick tree.
- Compress intervals including $r + 1$.
- Dynamic compression via ordered maps if online.

16. Related Algorithms/Data Structures

- Fenwick tree often needs compressed indices.
- Sweep line uses sorted events/compressed coordinates.
- Frequency arrays become possible after compression.

17. Practice Problems

Easy

- Rank Transform of an Array, LeetCode, compression, Easy.
- Coordinate Compression, AtCoder Library Practice, compression, Easy.

Medium

- Count of Smaller Numbers After Self, LeetCode, compression + BIT, Medium/Hard.
- Number of Longest Increasing Subsequence variants, compression + BIT, Medium.

Hard

- Count of Range Sum, LeetCode, prefix compression + BIT, Hard.
- Rectangle Area II, LeetCode, sweep + compression, Hard.

18. Interview Explanation

Coordinate compression replaces large values with ranks from the sorted unique list. It preserves order while allowing array-indexed data structures.

19. Revision Notes

- Sort unique.
- Rank via `lower_bound`.
- Preserve order, not distance.
- Trap: include all relevant coordinates.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Large sparse values	rank mapping	$O(n \log n)$	sorted unique + <code>lower_bound</code>	duplicates, missing query coords

Sweep Line Basics

1. Overview

Sweep line processes events sorted by coordinate/time to track active intervals or changes.

2. Intuition

Move a vertical line from left to right. At each event, intervals start or end, and the active count/state changes.

3. When to Use It

- Overlaps among intervals.
- Maximum active meetings/events.
- Trigger phrases: "maximum overlap", "number of active intervals", "timeline".

4. When Not to Use It

- Need actual merged intervals only: merge intervals is simpler.
- Need online updates: use trees.

- Small coordinate range: difference array may be simpler.

5. Core Concepts

- Start event adds **+1**, end event adds **-1**.
- Tie-breaking matters: whether intervals are closed or half-open.
- Active count tracks current overlap.

6. Step-by-Step Algorithm

1. Convert intervals into events.
2. Sort events by coordinate, with correct tie-breaking.
3. Scan events and update active state.
4. Update answer after each event.

7. Dry Run

Intervals: **[[1,4],[2,5],[7,9]]**.

Events: **(1,+1),(4,-1),(2,+1),(5,-1),(7,+1),(9,-1)**.

coord	delta	active	best
1	+1	1	1
2	+1	2	2
4	-1	1	2
5	-1	0	2
7	+1	1	2
9	-1	0	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int maxOverlap(const vector<pair<int,int>>& intervals) {
    vector<pair<int,int>> events;
    for (auto [l, r] : intervals) {
        events.push_back({l, +1});
        events.push_back({r, -1}); // half-open [l, r)
    }
    sort(events.begin(), events.end());
    int active = 0, best = 0;
    for (auto [coord, delta] : events) {
        active += delta;
        best = max(best, active);
    }
}
```

```
        return best;
    }

    int main() {
        vector<pair<int,int>> intervals = {{1,4},{2,5},{7,9}};
        cout << maxOverlap(intervals) << "\n";
    }
```

pYTHON IMPLEMENTATION

```
def max_overlap(intervals):
    events = []
    for l, r in intervals:
        events.append((l, 1))
        events.append((r, -1)) # half-open [l, r)
    events.sort()
    active = best = 0
    for _, delta in events:
        active += delta
        best = max(best, active)
    return best
```

10. Code Explanation

Every interval contributes a start and end event. Sorting events simulates the timeline. The active count is the number of intervals currently covering the sweep position.

11. Complexity Analysis

Operation	Time	Space
Build events	$O(n)$	$O(n)$
Sort events	$O(n \log n)$	$O(n)$
Scan	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Meeting rooms	max simultaneous intervals	sweep active count	LeetCode 253
Car pooling	capacity over route	sweep/diff	LeetCode 1094
Calendar bookings	max overlap	ordered sweep map	LeetCode 732

13. Common Mistakes

- Wrong tie-breaking for same coordinate.

- Confusing closed $[1, r]$ with half-open $[1, r)$.
- Updating answer before applying correct event order.

14. Edge Cases

- Same start/end.
- Touching intervals.
- Negative coordinates.
- Empty intervals.
- Many duplicate events.

15. Variations

- Ordered map sweep for sparse coordinates.
- Coordinate-compressed sweep.
- 2D sweep with segment tree.

16. Related Algorithms/Data Structures

- Difference array is sweep line on small integer coordinates.
- Merge intervals gives interval union, not active count.
- Heaps solve meeting rooms by end times.

17. Practice Problems

Easy

- Meeting Rooms, LeetCode/LintCode, interval conflict, Easy.
- Maximum Population Year, LeetCode, sweep/diff, Easy.

Medium

- Meeting Rooms II, LeetCode, sweep/heap, Medium.
- Car Pooling, LeetCode, sweep events, Medium.
- My Calendar I, LeetCode, interval checks, Medium.

Hard

- My Calendar III, LeetCode, sweep map, Hard.
- Rectangle Area II, LeetCode, 2D sweep, Hard.

18. Interview Explanation

Sweep line turns intervals into sorted start/end events. Scanning those events maintains the active intervals and lets me compute overlaps or coverage efficiently.

19. Revision Notes

- Start $+1$, end -1 .
- Tie-breaking depends on interval definition.

- $O(n \log n)$.
- Trap: closed vs half-open intervals.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Overlap/coverage over line	sort events	$O(n \log n)$	active += delta	same coordinate, interval type

Mo's Algorithm

1. Overview

Mo's algorithm answers offline range queries on static arrays by ordering queries to minimize pointer movement.

2. Intuition

If each query needs a current range $[L, R]$, moving from one query to a nearby query is cheaper than rebuilding from scratch. Sort queries in block order so nearby ranges are processed together.

3. When to Use It

- Many offline range queries.
- Static array.
- Add/remove one element can update answer quickly.
- Trigger phrases: "offline queries", "range distinct count", "query $[l, r]$ many times".

4. When Not to Use It

- Online queries required.
- Updates are frequent unless using advanced Mo with modifications.
- Query answer cannot be updated by add/remove efficiently.
- Simple prefix sum solves it.

5. Core Concepts

- Block size often $\sqrt{n}$.
- Sort by $(L/\text{block}, R)$ with odd-even optimization.
- Maintain current $[\text{curL}, \text{curR}]$ and answer.

6. Step-by-Step Algorithm

1. Store queries with original indices.
2. Sort queries by Mo order.
3. Initialize empty current range.
4. Move curL and curR to match each query using add/remove.

5. Save answer in original query order.

7. Dry Run

Array `[1,2,1,3]`, distinct count queries `[0,2]`, `[1,3]`.

query	operations	freq	answer
<code>[0,2]</code>	add 1,2,1	<code>{1:2,2:1}</code>	2
<code>[1,3]</code>	remove index 0, add 3	<code>{1:1,2:1,3:1}</code>	3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Query {
    int l, r, idx;
};

vector<int> distinctCountQueries(const vector<int>& a, vector<Query> queries) {
    int n = (int)a.size();
    int block = max(1, (int)sqrt(n));
    sort(queries.begin(), queries.end(), [&](const Query& x, const Query& y) {
        int bx = x.l / block, by = y.l / block;
        if (bx != by) return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });

    unordered_map<int,int> freq;
    vector<int> ans(queries.size());
    int curL = 0, curR = -1, distinct = 0;

    auto add = [&](int pos) {
        if (++freq[a[pos]] == 1) distinct++;
    };
    auto remove = [&](int pos) {
        if (--freq[a[pos]] == 0) distinct--;
    };

    for (auto q : queries) {
        while (curL > q.l) add(--curL);
        while (curR < q.r) add(++curR);
        while (curL < q.l) remove(curL++);
        while (curR > q.r) remove(curR--);
        ans[q.idx] = distinct;
    }
    return ans;
}

int main() {
```

```
vector<int> a = {1,2,1,3};
vector<Query> queries = {{0,2,0},{1,3,1}};
for (int x : distinctCountQueries(a, queries)) cout << x << " ";
}
```

pYTHON IMPLEMENTATION

```
from collections import defaultdict
from math import sqrt

def distinct_count_queries(a, queries):
    n = len(a)
    block = max(1, int(sqrt(n)))
    indexed = [(l, r, i) for i, (l, r) in enumerate(queries)]
    indexed.sort(key=lambda q: (q[0] // block, q[1] if (q[0] // block) % 2 == 0
    else -q[1]))

    freq = defaultdict(int)
    ans = [0] * len(queries)
    cur_l, cur_r, distinct = 0, -1, 0

    def add(pos):
        nonlocal distinct
        freq[a[pos]] += 1
        if freq[a[pos]] == 1:
            distinct += 1

    def remove(pos):
        nonlocal distinct
        freq[a[pos]] -= 1
        if freq[a[pos]] == 0:
            distinct -= 1

    for l, r, idx in indexed:
        while cur_l > l:
            cur_l -= 1
            add(cur_l)
        while cur_r < r:
            cur_r += 1
            add(cur_r)
        while cur_l < l:
            remove(cur_l)
            cur_l += 1
        while cur_r > r:
            remove(cur_r)
            cur_r -= 1
        ans[idx] = distinct
    return ans
```

10. Code Explanation

Queries are sorted so current range changes gradually. **add** and **remove** update frequency counts and the number of distinct values in $O(1)$ average time.

11. Complexity Analysis

Operation	Time	Space
Sort queries	$O(q \log q)$	$O(q)$
Pointer movement	$O((n + q) \sqrt{n})$ typical	$O(n)$
Add/remove	$O(1)$ average	$O(\text{distinct})$

12. Common Patterns

Pattern	Identify	Approach	Examples
Range distinct	static array many queries	Mo + frequency	SPOJ DQUERY
Powerful array	answer changes by freq	Mo	Codeforces 86D
Range mode-ish	offline frequency	Mo	CP tasks

13. Common Mistakes

- Forgetting original query index.
- Bad add/remove symmetry.
- Using Mo when prefix/sparse table is enough.
- Off-by-one inclusive ranges.

14. Edge Cases

- Query length 1.
- Repeated values.
- Large values needing compression/hash.
- No queries.
- All queries same range.

15. Variations

- Mo with updates/modifications.
- Tree Mo using Euler tour.
- Hilbert order optimization.

16. Related Algorithms/Data Structures

- Offline queries are the broad category.
- Fenwick tree solves many offline order/rank queries.
- Segment tree solves online associative range queries.

17. Practice Problems

Easy

- DQUERY, SPOJ, distinct count, Easy/Medium.
- Range Frequency Queries, CP practice, Mo basics, Easy.

Medium

- Powerful Array, Codeforces 86D, frequency contribution, Medium.
- COT2, SPOJ, tree Mo, Medium/Hard.

Hard

- Xor and Favorite Number, Codeforces 617E, prefix xor + Mo, Hard.
- Dynamic range query variants, Codeforces, Mo with updates, Hard.

18. Interview Explanation

Mo's algorithm is for offline static range queries where I can add or remove one boundary element efficiently. Sorting queries by blocks reduces total pointer movement.

19. Revision Notes

- Offline only.
- Sort by L block and R.
- Maintain current range.
- Trap: inclusive boundaries.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Static range queries with custom add/remove	reorder queries	$\sim O((n+q)\sqrt{n})$	move current [L,R]	index restore, freq bugs

Offline Queries on Arrays

1. Overview

Offline query processing reorders queries or preprocesses all inputs together to answer faster than online processing.

2. Intuition

If answers do not need to be returned immediately, sort queries by useful property and process them with a data structure that changes monotonically.

3. When to Use It

- All queries known in advance.

- Query order does not matter for computation.
- Trigger phrases: "answer q queries", "offline allowed", "static array", "threshold".

4. When Not to Use It

- Interactive/online systems.
- Each query depends on previous answer.
- Simpler prefix/sparse table handles it directly.

5. Core Concepts

- Sort queries by threshold, right endpoint, or block.
- Preserve original query index.
- Use Fenwick/segment tree/DSU/frequency structure while scanning.

6. Step-by-Step Algorithm

1. Attach index to every query.
2. Sort array events and queries by chosen key.
3. Maintain a data structure as the key increases.
4. Answer each query from current structure.
5. Write result to original position.

7. Dry Run

Count elements $\leq x$ in range $[1, r]$.

Array values with indices sorted: $(1, 2), (3, 0), (5, 1)$. Queries sorted by x .

query	add values $\leq x$ to BIT	answer
$[0, 2], x=3$	add idx 2 and 0	2
$[1, 2], x=5$	add idx 1	2

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
    void add(int idx, int val) {
        for (++idx; idx <= n; idx += idx & -idx) bit[idx] += val;
    }
    int sumPrefix(int idx) {
        int res = 0;
        for (++idx; idx > 0; idx -= idx & -idx) res += bit[idx];
    }
};
```

```

        return res;
    }
    int rangeSum(int l, int r) {
        return sumPrefix(r) - (l ? sumPrefix(l - 1) : 0);
    }
};

struct Query { int l, r, x, idx; };

vector<int> countLessEqualInRange(const vector<int>& a, vector<Query> queries) {
    int n = (int)a.size();
    vector<pair<int,int>> values;
    for (int i = 0; i < n; i++) values.push_back({a[i], i});
    sort(values.begin(), values.end());
    sort(queries.begin(), queries.end(), [](auto& p, auto& q) {
        return p.x < q.x;
    });
    Fenwick fw(n);
    vector<int> ans(queries.size());
    int ptr = 0;
    for (auto q : queries) {
        while (ptr < n && values[ptr].first <= q.x) {
            fw.add(values[ptr].second, 1);
            ptr++;
        }
        ans[q.idx] = fw.rangeSum(q.l, q.r);
    }
    return ans;
}

int main() {
    vector<int> a = {3,5,1};
    vector<Query> qs = {{0,2,3,0},{1,2,5,1}};
    for (int x : countLessEqualInRange(a, qs)) cout << x << " ";
}

```

pYTHON IMPLEMENTATION

```

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, idx, val):
        idx += 1
        while idx <= self.n:
            self.bit[idx] += val
            idx += idx & -idx
    def prefix(self, idx):
        res = 0
        idx += 1
        while idx > 0:

```

```

        res += self.bit[idx]
        idx -= idx & -idx
    return res

def range_sum(self, l, r):
    return self.prefix(r) - (self.prefix(l - 1) if l else 0)

def count_less_equal_in_range(a, queries):
    values = sorted((x, i) for i, x in enumerate(a))
    indexed = sorted((x, l, r, i) for i, (l, r, x) in enumerate(queries))
    fw, ans, ptr = Fenwick(len(a)), [0] * len(queries), 0
    for x, l, r, idx in indexed:
        while ptr < len(values) and values[ptr][0] <= x:
            fw.add(values[ptr][1], 1)
            ptr += 1
        ans[idx] = fw.range_sum(l, r)
    return ans

```

10. Code Explanation

Queries are sorted by threshold x . As x increases, we add newly allowed array positions to Fenwick. A range sum then counts active positions inside $[l, r]$.

11. Complexity Analysis

Operation	Time	Space
Sort values/queries	$O((n+q) \log(n+q))$	$O(n+q)$
Each update/query	$O(\log n)$	$O(n)$
Overall	$O((n+q) \log n)$	$O(n+q)$

12. Common Patterns

Pattern	Identify	Approach	Examples
Threshold queries	values $\leq x$ in range	sort + BIT	SPOJ KQUERY
Distinct in range	last occurrence	sort by right + BIT	CSES Distinct Values Queries
Range custom stats	static, many queries	Mo's algorithm	SPOJ DQUERY

13. Common Mistakes

- Not restoring original order.
- Sorting by wrong key.
- Updating data structure too far or not far enough.
- 0-based/1-based Fenwick bugs.

14. Edge Cases

- No queries.

- Query threshold below all values.
- Threshold above all values.
- Duplicate values.
- Single-index ranges.

15. Variations

- Sort by right endpoint for distinct counts.
- Parallel binary search for answer search queries.
- CDQ divide and conquer for offline dominance.

16. Related Algorithms/Data Structures

- Mo's algorithm is an offline range-query method.
- Fenwick tree is common for offline counting.
- Sweep line is offline event processing on coordinates.

17. Practice Problems

Easy

- Static Range Queries with Threshold, GFG, sort + BIT, Easy/Medium.
- Distinct Values Queries, CSES, offline BIT, Easy/Medium.

Medium

- KQUERY, SPOJ, offline threshold + BIT, Medium.
- Count of Smaller Numbers After Self, LeetCode, offline/BIT, Medium/Hard.

Hard

- Count of Range Sum, LeetCode, offline compression + BIT, Hard.
- Dynamic Connectivity Offline, Codeforces style, DSU rollback, Hard.

18. Interview Explanation

Offline processing uses the fact that all queries are known. I sort them by a helpful key, process array events incrementally, and store answers back by original query index.

19. Revision Notes

- Attach original index.
- Sort queries/events.
- Monotonic processing + DS.
- Trap: answer order.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Known-in-advance queries	sort + DS	often $O((n+q)\log n)$	process events once	duplicates, original index

Divide and Conquer Optimization on Arrays

1. Overview

Divide and conquer optimization speeds up certain DP transitions when the optimal split point is monotonic.

2. Intuition

For DP like $dp[g][mid] = \min \text{ over } k < mid \text{ of } dp[g-1][k] + \text{cost}(k+1, mid)$, if the best k for later mid never moves left, we can search a restricted split range recursively.

3. When to Use It

- DP over partitions/groups.
- Transition is $O(n)$ per state, causing $O(k n^2)$.
- Optimal split is monotonic.
- Trigger phrases: "partition array into k groups", "minimize cost", "divide and conquer DP optimization".

4. When Not to Use It

- Monotonicity does not hold.
- Cost cannot be computed quickly.
- Simpler $O(kn^2)$ passes constraints.
- Convex hull trick or Knuth optimization may fit better.

5. Core Concepts

- State: $dp[layer][i]$.
- Transition: choose split j .
- Monotone opt: $opt[layer][i] \leq opt[layer][i+1]$.
- Recursive solve range $[l, r]$ with candidate opt range $[optL, optR]$.

6. Step-by-Step Algorithm

1. Compute base DP.
2. For each layer, recursively solve $dpCur[l..r]$.
3. At midpoint, try candidate splits only from $[optL, optR]$.
4. Record best split.
5. Recurse left with $[optL, bestSplit]$.
6. Recurse right with $[bestSplit, optR]$.

7. Dry Run

For layer g , solve indices $[1, 8]$.

recursive range	mid	candidate opt range	found best
[1,8]	4	[0,7]	2
[1,3]	2	[0,2]	1
[5,8]	6	[2,7]	4

Monotonicity narrows future candidate ranges.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = (1LL << 62);

// Example cost: sum of a[1..r]^2, using 1-based prefix sums.
long long cost(const vector<long long>& pref, int l, int r) {
    long long s = pref[r] - pref[l - 1];
    return s * s;
}

void computeLayer(
    int left, int right, int optLeft, int optRight,
    const vector<long long>& prev,
    vector<long long>& cur,
    const vector<long long>& pref
) {
    if (left > right) return;
    int mid = (left + right) / 2;
    pair<long long, int> best = {INF, -1};

    int upper = min(optRight, mid - 1);
    for (int split = optLeft; split <= upper; split++) {
        long long value = prev[split] + cost(pref, split + 1, mid);
        if (value < best.first) best = {value, split};
    }
    cur[mid] = best.first;
    int opt = best.second;

    computeLayer(left, mid - 1, optLeft, opt, prev, cur, pref);
    computeLayer(mid + 1, right, opt, optRight, prev, cur, pref);
}

long long partitionCost(vector<int> a, int groups) {
    int n = (int)a.size();
    vector<long long> pref(n + 1, 0);
    for (int i = 1; i <= n; i++) pref[i] = pref[i - 1] + a[i - 1];

    vector<long long> prev(n + 1, INF), cur(n + 1, INF);
```

```

    prev[0] = 0;
    for (int g = 1; g <= groups; g++) {
        fill(cur.begin(), cur.end(), INF);
        computeLayer(1, n, 0, n - 1, prev, cur, pref);
        prev.swap(cur);
    }
    return prev[n];
}

int main() {
    vector<int> a = {1,2,3,4};
    cout << partitionCost(a, 2) << "\n";
}

```

pYTHON IMPLEMENTATION

```

INF = 10**30

def partition_cost(a, groups):
    n = len(a)
    pref = [0] * (n + 1)
    for i, x in enumerate(a, 1):
        pref[i] = pref[i - 1] + x

    def cost(l, r):
        s = pref[r] - pref[l - 1]
        return s * s

    def compute(left, right, opt_left, opt_right, prev, cur):
        if left > right:
            return
        mid = (left + right) // 2
        best_value, best_split = INF, -1
        for split in range(opt_left, min(opt_right, mid - 1) + 1):
            value = prev[split] + cost(split + 1, mid)
            if value < best_value:
                best_value, best_split = value, split
        cur[mid] = best_value
        compute(left, mid - 1, opt_left, best_split, prev, cur)
        compute(mid + 1, right, best_split, opt_right, prev, cur)

    prev = [INF] * (n + 1)
    prev[0] = 0
    for _ in range(groups):
        cur = [INF] * (n + 1)
        compute(1, n, 0, n - 1, prev, cur)
        prev = cur
    return prev[n]

```

10. Code Explanation

Each layer represents using one more group. `computeLayer` calculates DP values for a range of endpoints. It tests splits only inside the allowed optimal range and uses the midpoint's best split to restrict recursive calls.

11. Complexity Analysis

Operation	Time	Space
Naive DP	$O(\text{groups} * n^2)$	$O(n)$ or $O(\text{groups} * n)$
D&C optimized DP	$O(\text{groups} * n \log n)$ to $O(\text{groups} * n)$ depending on analysis/cost	$O(n)$
Cost query	$O(1)$ if precomputed	$O(n)$ if not optimized

12. Common Patterns

Pattern	Identify	Approach	Examples
Partition DP	split array into k groups	D&C DP optimization	Codeforces Ciel and Gondolas
Clustering cost	group contiguous items	monotone opt DP	AtCoder DP optimizations
Batch scheduling	ordered jobs into groups	optimized transition	CP tasks

13. Common Mistakes

- Applying without proving monotone opt.
- Slow cost function ruining optimization.
- Invalid split range when `split >= mid`.
- Confusing with divide-and-conquer algorithm, not DP optimization.

14. Edge Cases

- `groups > n`.
- Empty prefix state.
- Impossible states as INF.
- Overflow in cost.
- One group.

15. Variations

- Knuth optimization for stronger quadrangle conditions.
- Convex hull trick for linear transitions.
- Aliens trick/parametric search for group constraints.

16. Related Algorithms/Data Structures

- Prefix sums make cost queries $O(1)$.
- Segment tree may optimize other DP transitions.
- Convex hull trick is chosen when transition has linear form.

17. Practice Problems

Easy

- Partition Array for Maximum Sum, LeetCode, basic partition DP, Easy/Medium.
- Split Array Largest Sum, LeetCode, partition intuition/binary search, Easy/Medium.

Medium

- Allocate Books, GFG, partition + binary search, Medium.
- AtCoder DP contest partition-style tasks, AtCoder, DP transition, Medium.

Hard

- Ciel and Gondolas, Codeforces 321E, D&C DP, Hard.
- Yet Another Minimization Problem, Codeforces, D&C DP, Hard.
- IOI/CEOI batch scheduling variants, CP, optimized DP, Hard.

18. Interview Explanation

Divide and conquer DP optimization speeds up partition DP when the best split point moves monotonically. I compute the midpoint first, find its best split, and use that split to narrow the search for left and right halves.

19. Revision Notes

- Applies to DP split transitions.
- Need monotone opt.
- Cost must be fast.
- Trap: using it without proof.

20. Final Cheat Sheet

Use	Operation	Complexity	Code idea	Edge cases
Monotone partition DP	recursive layer compute	about $O(k n \log n)$ or better	solve mid, restrict opt	impossible states, overflow